A+ Plane

Analytics Dashboard

Feature Specification & Vibe-Coding Prompt

A professional English implementation guide for the analytics feature in the A+ Plane college grievance portal.

Feature	Analytics Dashboard
GitHub folder	features/analytics/
Primary users	Authorized administrators
Core metrics	Total, pending, and resolved complaints; department-wise charts; average resolution time; complaint trends

Scope note: This document turns the four requested analytics requirements into an implementation specification and reusable AI coding prompt. Exact existing routes, database fields, and component conventions must be verified in the repository before implementation.

1. Feature Overview

The A+ Plane Analytics Dashboard should help authorized administrators understand complaint volume, current workload, department distribution, resolution performance, and changes over time. It must use real complaint data from the existing backend and database rather than hardcoded demonstration values.

Goals

- Provide a clear summary of total, pending, and resolved complaints.
- Visualize complaint distribution by department or category using readable charts.
- Calculate average resolution time from valid submission and resolution timestamps.
- Display complaint trends over a selectable time period.
- Support administrative decision-making while respecting role-based access and student privacy.

Suggested access

The dashboard is intended for authenticated administrators. The backend must enforce admin authorization for analytics endpoints; hiding a link in the frontend is not sufficient.

2. Functional Requirements

2.1 Complaint Summary Cards

- **Total Complaints:** Count of complaints matching the selected date/filter scope.
- **Pending Complaints:** Count complaints in statuses treated as pending by the actual backend status model. Define the exact status mapping from the existing enum; do not guess.
- **Resolved Complaints:** Count complaints that meet the system's actual resolved-status definition.
- Show loading, empty, and error states. If data cannot be fetched, display an error rather than zero as if it were a real result.
- Ensure counts respond consistently to any selected date range or department filter.

2.2 Department-Wise Charts

- Show complaint counts grouped by department or category according to the actual data model.
- Use a bar chart for comparing counts; a donut/pie chart may be added if it improves readability.
- Label axes, categories, values, and the selected time period clearly.
- Provide a useful empty state when no department/category data exists.
- Do not label a grievance category as an academic department unless the database actually distinguishes those concepts.

2.3 Average Resolution Time

- Calculate the duration between a complaint's creation/submission timestamp and its actual resolution timestamp.
- Include only complaints with valid timestamps that qualify as resolved under the actual status model.
- Define the unit clearly, such as hours or days, and keep it consistent throughout the dashboard.
- Handle missing, invalid, or out-of-order timestamps safely; do not silently count unresolved complaints as zero-duration resolutions.
- If the current schema does not store a reliable resolution timestamp, identify that gap and propose a compatible, non-destructive schema change.

2.4 Complaint Trends

- Display complaint volume over time, grouped by day, week, or month according to the selected date range.

- Use the complaint submission/creation timestamp for submitted-volume trends unless a different metric is explicitly defined.
- Make date range and grouping clear to the administrator.
- Return consistent time buckets, including zero-count buckets where practical, so charts do not misrepresent missing dates.
- Handle timezone boundaries consistently between backend aggregation and frontend display.

3. Data and Calculation Rules

Metric	Definition to implement
Total complaints	Count of complaints in the selected scope, using the agreed date filter.
Pending complaints	Count of statuses mapped to pending by the current complaint-status workflow.
Resolved complaints	Count of complaints whose status qualifies as resolved under the existing workflow.
Department/category distribution	Group complaint counts by the actual department/category field; keep the label consistent with the schema.
Average resolution time	Average of (resolution timestamp – submission timestamp) for qualifying resolved complaints with valid timestamps.
Complaint trends	Counts grouped into time buckets using the submission/creation timestamp, unless the product explicitly defines another trend.

> Important: the current project schema and status enum must be inspected. For example, if the schema has a general updatedAt timestamp but no dedicated resolvedAt timestamp or status history, updatedAt must not automatically be treated as the resolution time unless the workflow guarantees that meaning.

4. Suggested Folder Structure

Create the feature in the requested GitHub folder, following the repository's existing architecture and naming conventions. The following is a proposal; adapt it after inspecting the project.

```
features/
  analytics/
    components/
      AnalyticsSummaryCards.jsx
      DepartmentComplaintsChart.jsx
      ComplaintTrendsChart.jsx
      ResolutionTimeCard.jsx
      AnalyticsFilters.jsx
    pages/
      AnalyticsDashboard.jsx
    services/
      analyticsApi.js
    utils/
      analyticsFormatters.js
```

> If the repository keeps all React pages under frontend/src/pages or all API clients under frontend/src/services, integrate this feature there instead of creating a disconnected second frontend structure. The requested features/analytics/ folder can contain the feature module if the project architecture supports it.

5. Backend API Requirements

The exact route names must be aligned with the existing backend. The following are suggested responsibilities, not confirmed existing endpoints.

- **Summary endpoint:** return total, pending, and resolved counts for the selected filter scope.
- **Department/category endpoint:** return grouped counts suitable for charts.
- **Resolution-time endpoint or summary field:** return the calculated average and unit, based on reliable resolution timestamps.
- **Trend endpoint:** return time buckets and complaint counts for the selected range and grouping.
- All analytics endpoints must require authenticated admin access.
- Validate date range, grouping, and filter parameters on the server.
- Use database aggregation where appropriate and avoid returning private complaint descriptions or unnecessary student identifiers.
- Use a consistent response format and appropriate HTTP status codes.

6. Master Vibe-Coding Prompt

Copy this prompt into your AI coding assistant while the existing A+ Plane repository is open.

```
You are a senior full-stack engineer helping implement the Analytics Dashboard feature for A+ Plane, a college grievance port

FEATURE FOLDER
features/analytics/

REQUIRED DASHBOARD
1. Total complaints.
2. Pending complaints.
3. Resolved complaints.
4. Department-wise or category-wise complaint charts, using the field that actually exists in the database.
5. Average complaint resolution time.
6. Complaint trends over time.

FIRST INSPECT THE PROJECT
- Inspect the repository structure, existing React frontend, route configuration, API service, admin dashboard, Express route
- Reuse the current design system, authentication, API client, chart library, and folder conventions.
- Do not create a disconnected duplicate frontend or duplicate API client.
- Confirm whether the project architecture supports features/analytics/; if not, integrate it using the established conventio

DATA ACCURACY
- Use real backend/database data only. Do not hardcode sample counts or fake chart series.
- Define pending and resolved based on the actual status enum and business workflow.
- Group by department only if the schema has a department field. If it has only complaint categories, label the chart “Compla
- Calculate average resolution time from reliable submission and resolution timestamps for qualifying resolved complaints.
- If the schema lacks a reliable resolution timestamp, report the gap and propose a safe, non-destructive schema/migration ch
- Use submission/creation timestamps for complaint-volume trends unless a different definition is specified.
- Validate date filters and aggregation parameters on the backend.

SECURITY
- Make analytics available only to authenticated administrators.
- Enforce role authorization in the backend, not only by hiding frontend navigation.
- Do not return complaint descriptions, passwords, tokens, or unnecessary personal information in analytics responses.
- Never expose environment secrets in code or logs.

UI AND UX
- Build a clean, responsive dashboard with summary cards, labelled charts, date filters, loading states, empty states, and cl
- Make charts readable on mobile and desktop, with accessible labels and non-color-only distinctions where practical.
- Keep displayed metrics consistent with the selected filters and show the date range used.
- Do not display zero as a successful metric when the API request has failed.

IMPLEMENTATION
- Add the necessary analytics frontend components and API service.
- Add backend analytics routes/services only after inspecting the current architecture and avoiding duplicated logic.
- Use TypeScript/types or JSDoc conventions consistent with the repository.
- Handle invalid date ranges, empty datasets, timezone boundaries, and database errors.
- Preserve existing data and unrelated working features.

VERIFY
- Run the available lint, type-check, test, and production build commands.
- Test admin authorization, student denial, date filtering, empty data, status mapping, and resolution-time calculations wher
- Do not claim that tests passed unless they were actually run.
- Finish by listing the files created/changed, endpoints added or reused, calculation definitions, commands run, actual resul
```

7. Frontend Implementation Prompt

```
Implement the Analytics Dashboard frontend for A+ Plane.

- Inspect the existing AdminDashboard.jsx, App.jsx/routes, ProtectedRoute, auth state, API service, styling conventions, and
- Create the analytics page and components in features/analytics/ if that matches the repository architecture; otherwise use
- Add summary cards for total, pending, and resolved complaints.
- Add a department-wise chart only when a real department field exists. If the database only stores categories, use the title
- Add a complaint trend chart with a clear date range and appropriate day/week/month grouping.
- Add an average resolution-time metric with a clearly labelled unit.
- Connect to the real analytics API. Do not hardcode sample values or use mock data in production code.
- Add date filters only if they can be supported by the backend. Keep all metrics and charts synchronized with the selected f
- Show loading, empty, retry, and error states.
- Use accessible chart titles, labels, tooltips, and responsive containers.
- Protect the page using the existing route guard and verified admin role.
- Avoid adding a new chart dependency if a compatible library already exists.
- Run the production build and report the actual outcome.
```

8. Backend Implementation Prompt

```
Implement analytics aggregation for the existing A+ Plane Express/TypeScript/Prisma backend.

- Inspect the current complaint schema, status enum, timestamp fields, route registration, authentication middleware, authori
- Add an admin-only analytics route and service that return the summary counts, category/department distribution, complaint t
- Use the actual status enum to define pending and resolved. Document the mapping.
- Use database aggregation or efficient queries where appropriate; avoid loading every complaint into memory unnecessarily.
- Validate optional start date, end date, grouping, and category/department filters. Reject invalid or reversed date ranges.
- Calculate resolution duration only from reliable submission and resolution timestamps. If the schema has no reliable resolu
- Return aggregate values only; do not expose complaint descriptions or unnecessary student details.
- Require a verified admin role for every analytics endpoint.
- Use consistent response types, safe errors, and appropriate HTTP status codes.
- Test empty data, mixed statuses, date boundaries, timezone handling, invalid filters, and role restrictions where possible.
- Document actual routes and response examples from the implemented code, not guessed routes.
```

9. Example Response Contract

Use this as a design example only. The coding assistant must adapt it to the actual API conventions and must not claim that this endpoint already exists.

```
{
  "summary": {
    "totalComplaints": 0,
    "pendingComplaints": 0,
    "resolvedComplaints": 0,
    "averageResolutionHours": null
  },
  "distribution": [
    {
      "label": "Hostel",
      "count": 0
    }
  ],
  "trends": [
    {
      "period": "2026-10-01",
      "count": 0
    }
  ],
  "filters": {
    "startDate": null,
    "endDate": null,
    "groupBy": "day"
  }
}
```

The values above are placeholders for explaining a possible response shape, not actual A+ Plane metrics. A missing or uncalculable average should be represented explicitly rather than silently reported as zero.

10. Acceptance Criteria

- An authorized administrator can open the analytics dashboard; a student cannot access protected analytics data.
- Total, pending, and resolved counts are computed from real database records and the documented status mapping.
- Department-wise data is labelled accurately; categories are not misrepresented as departments.
- Average resolution time uses a reliable resolution timestamp and a clearly defined unit.
- Complaint trends use a documented timestamp and consistent date buckets.
- Date filters, if implemented, affect the cards and charts consistently.
- Loading, empty, error, and retry states work correctly.
- No sensitive complaint details or unnecessary personal data appear in aggregate responses.
- The existing application continues to build, and actual test results are recorded.
- The final report lists files changed, endpoints, data assumptions, and unresolved dependencies.

11. Vibe-Coding Evidence Checklist

- Save the exact prompt used with the AI coding assistant.
- Keep the AI conversation showing how the feature was refined after running the application.
- Use Git commits or a change log to record created and edited files.
- Capture screenshots of the running analytics dashboard, with sample/demo data clearly identified if used.
- Save actual lint/build/test output and API test results with private data removed.
- Record which features are complete, partial, or blocked by missing schema fields or endpoints.

Important: This PDF documents the specification and prompts. It is not proof by itself that the analytics feature is implemented, connected to live data, or tested. Use the repository history and actual test results to demonstrate the work.